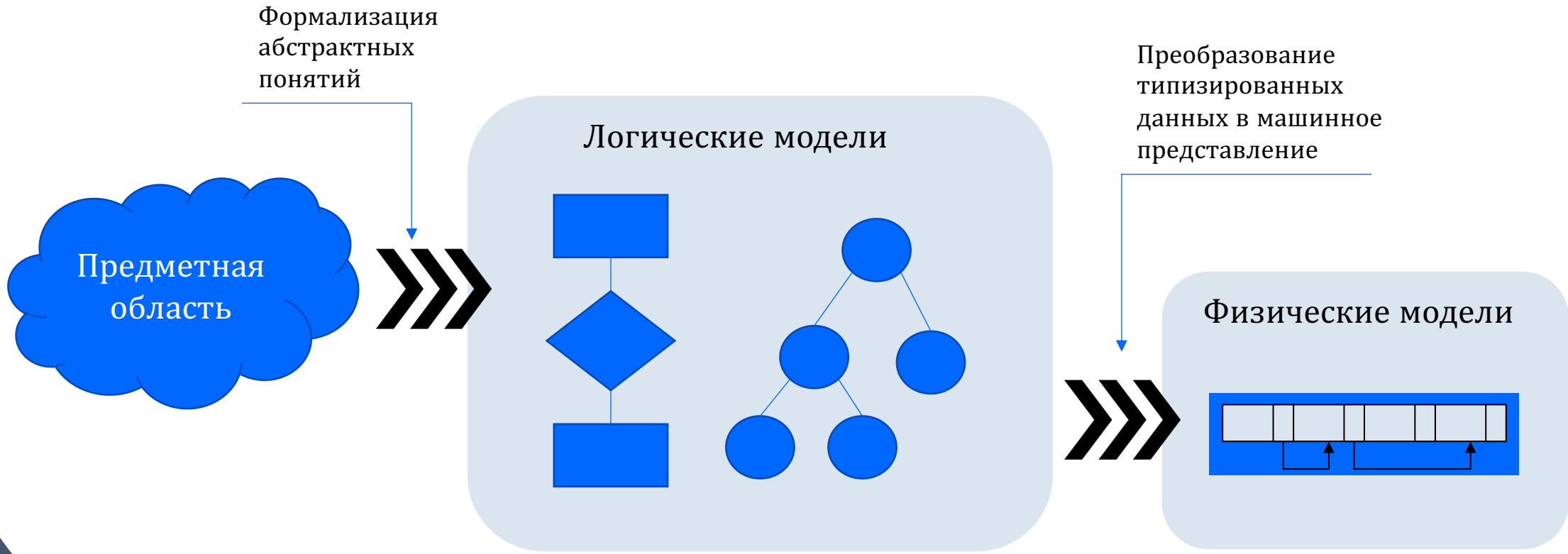
Многоуровневые модели предметной области. 12 правил Кодда. Знакомство с PostgreSQL

ст. преподаватель кафедры АСУ
Валова Анастасия Александровна

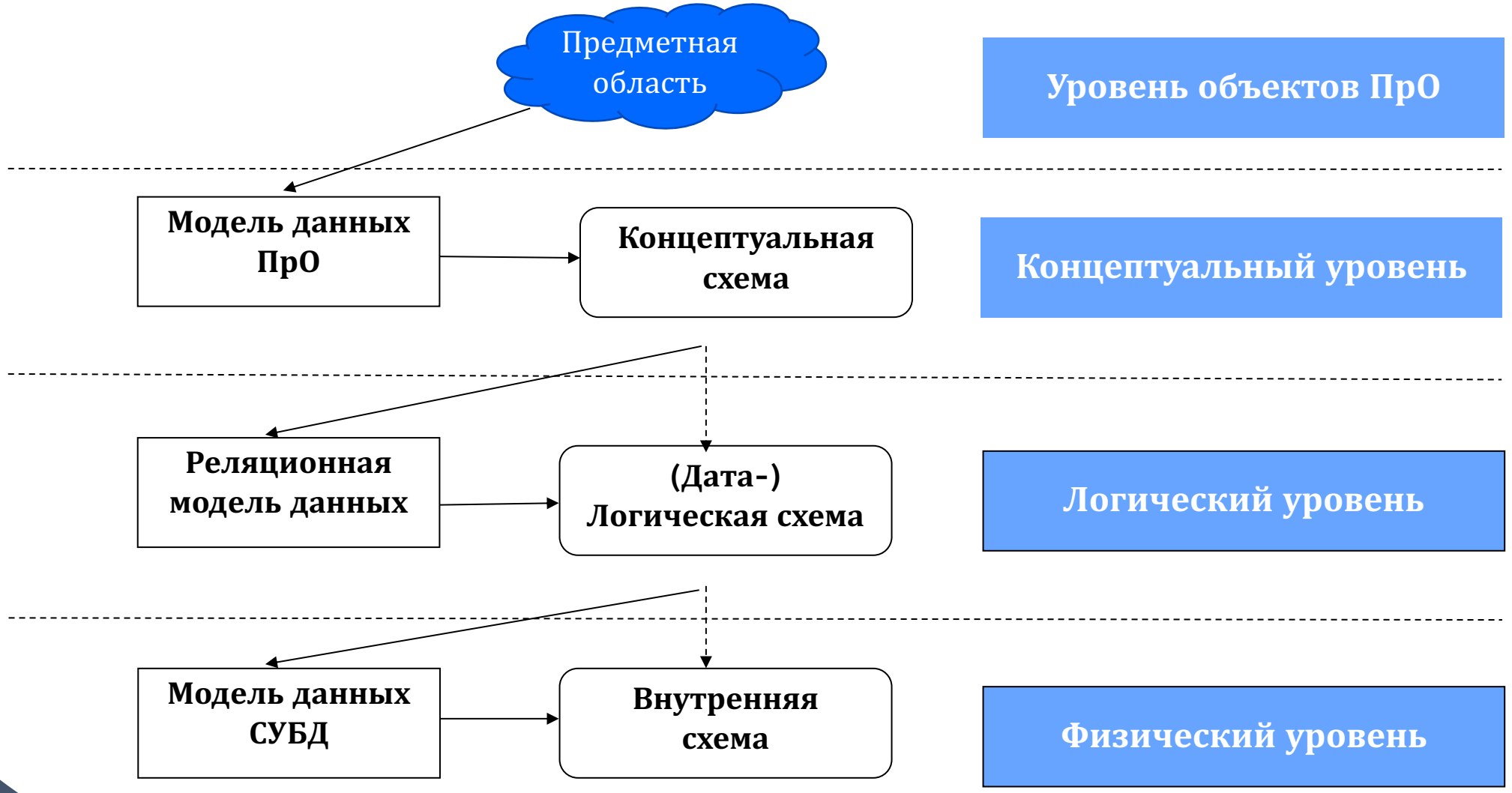
Предметная область и модели данных



Типология моделей представления информации

Концептуальный уровень	Инфологические модели
Логический уровень	Даталогические модели
Физический уровень	Физические модели

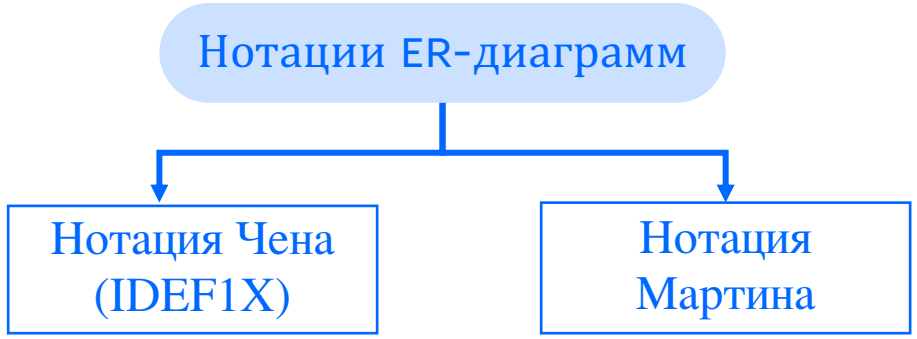
Поэтапное проектирование базы данных

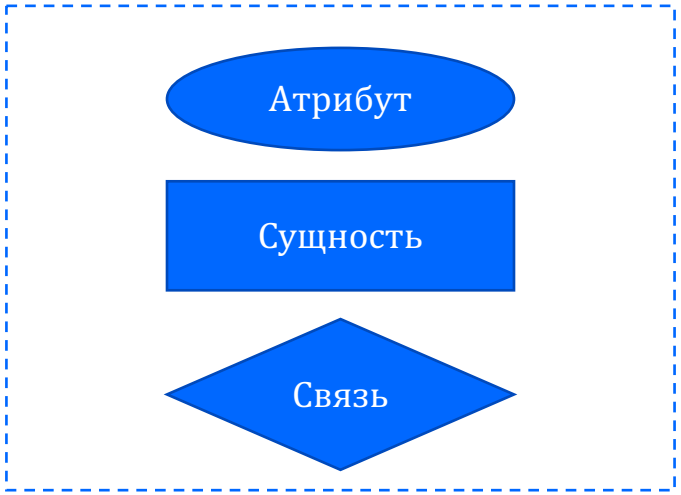


Концептуальная модель данных

Диаграмма сущность-связь (ER-диаграмма)

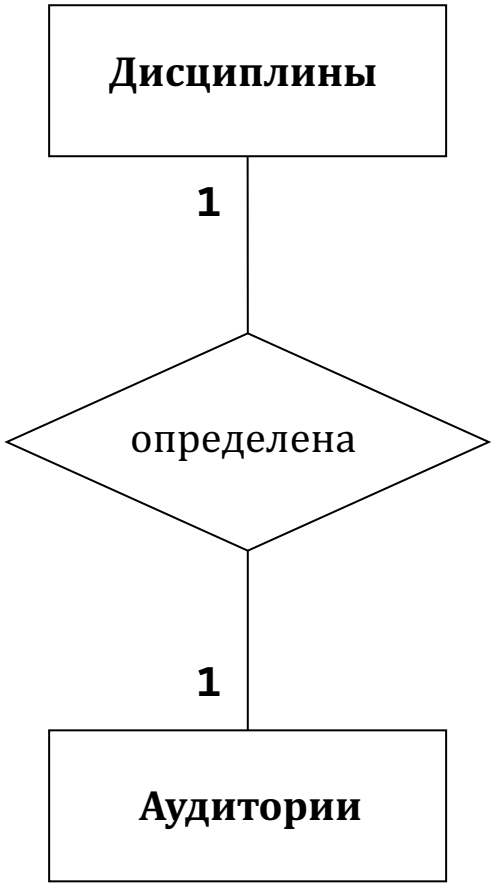
- **Не зависит от реализации:** Нет упоминаний о таблицах, столбцах, СУБД или способах хранения.
- **Ориентирована на человека:** Её цель — быть понятной для заказчиков, бизнес-аналитиков и разработчиков.
- **Отражает структуру бизнеса:** Описывает ключевые объекты (сущности), их характеристики (атрибуты) и логические связи между ними.

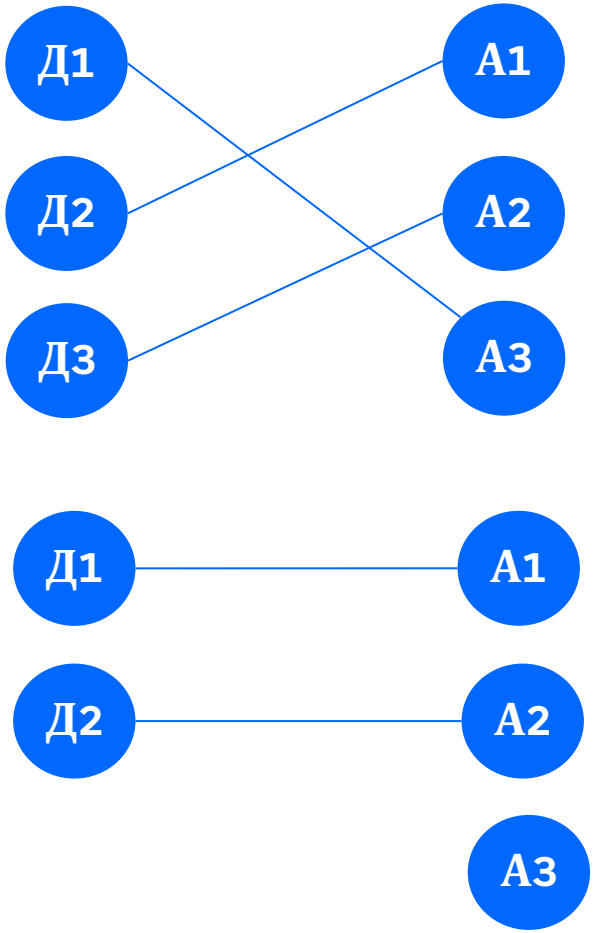




Типология связей объекта

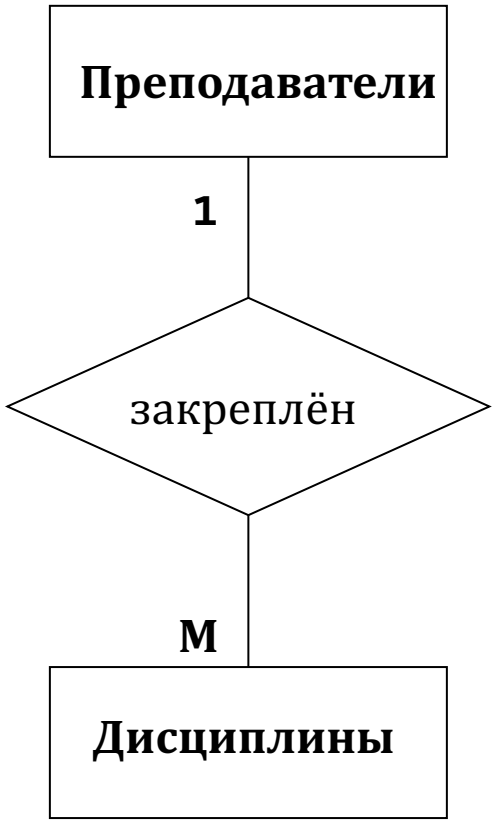
Связь мощности «один к одному» (1:1)

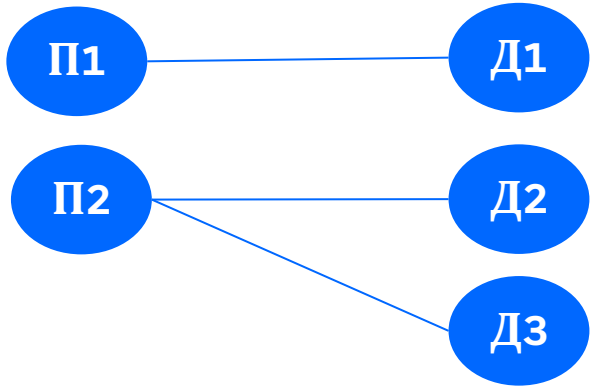


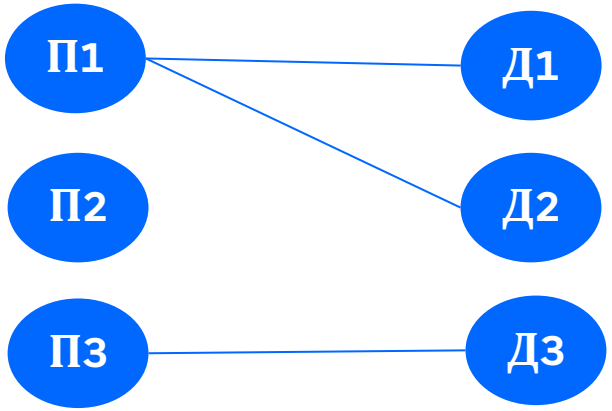


Типология связей объекта

Связь мощности «один ко многим» (1:M)

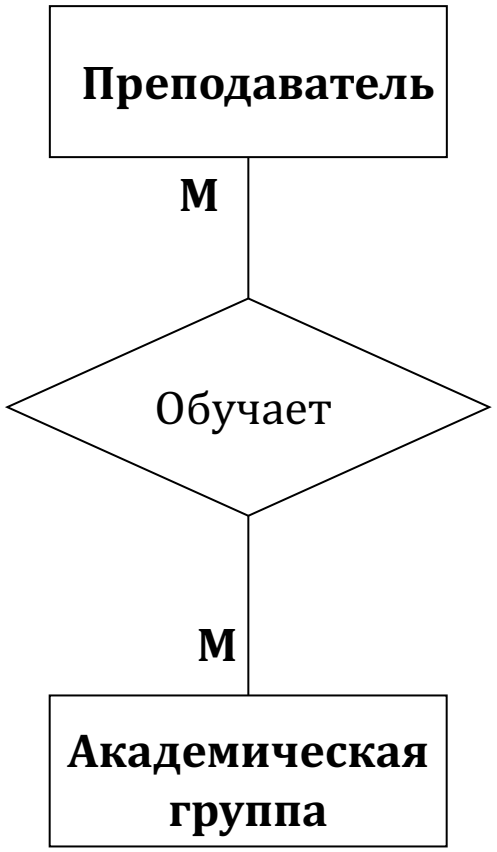


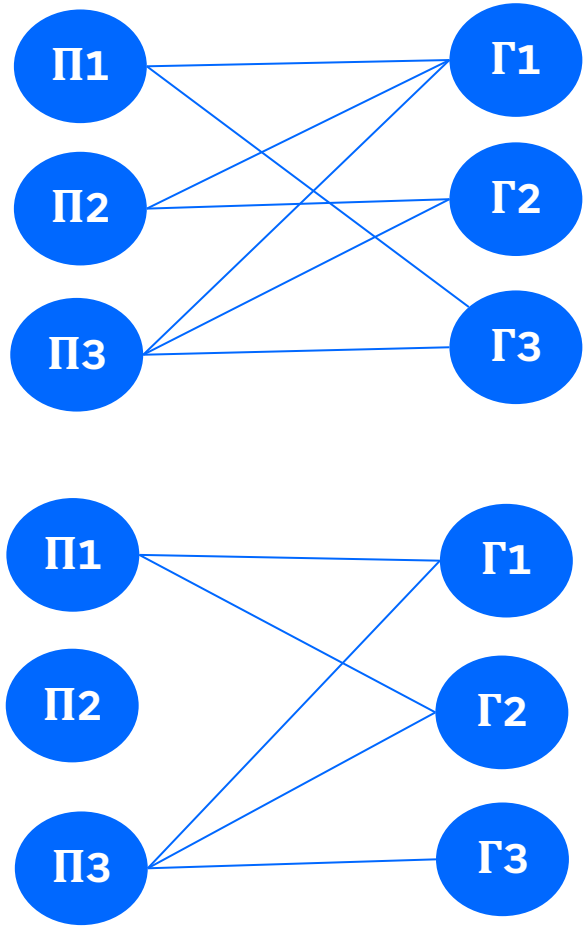




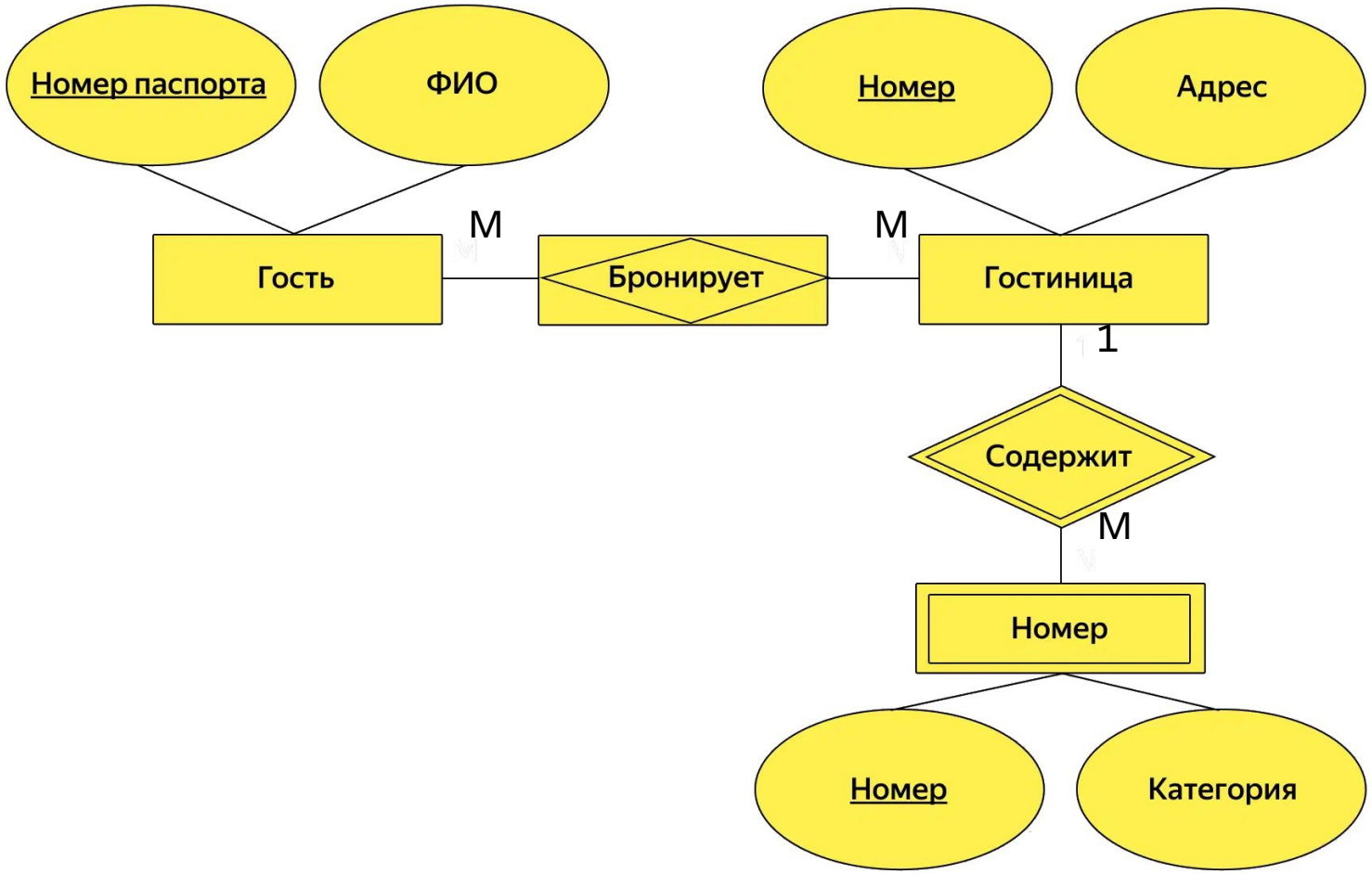
Типология связей объекта

Связь мощности «многие ко многим» (N:N)

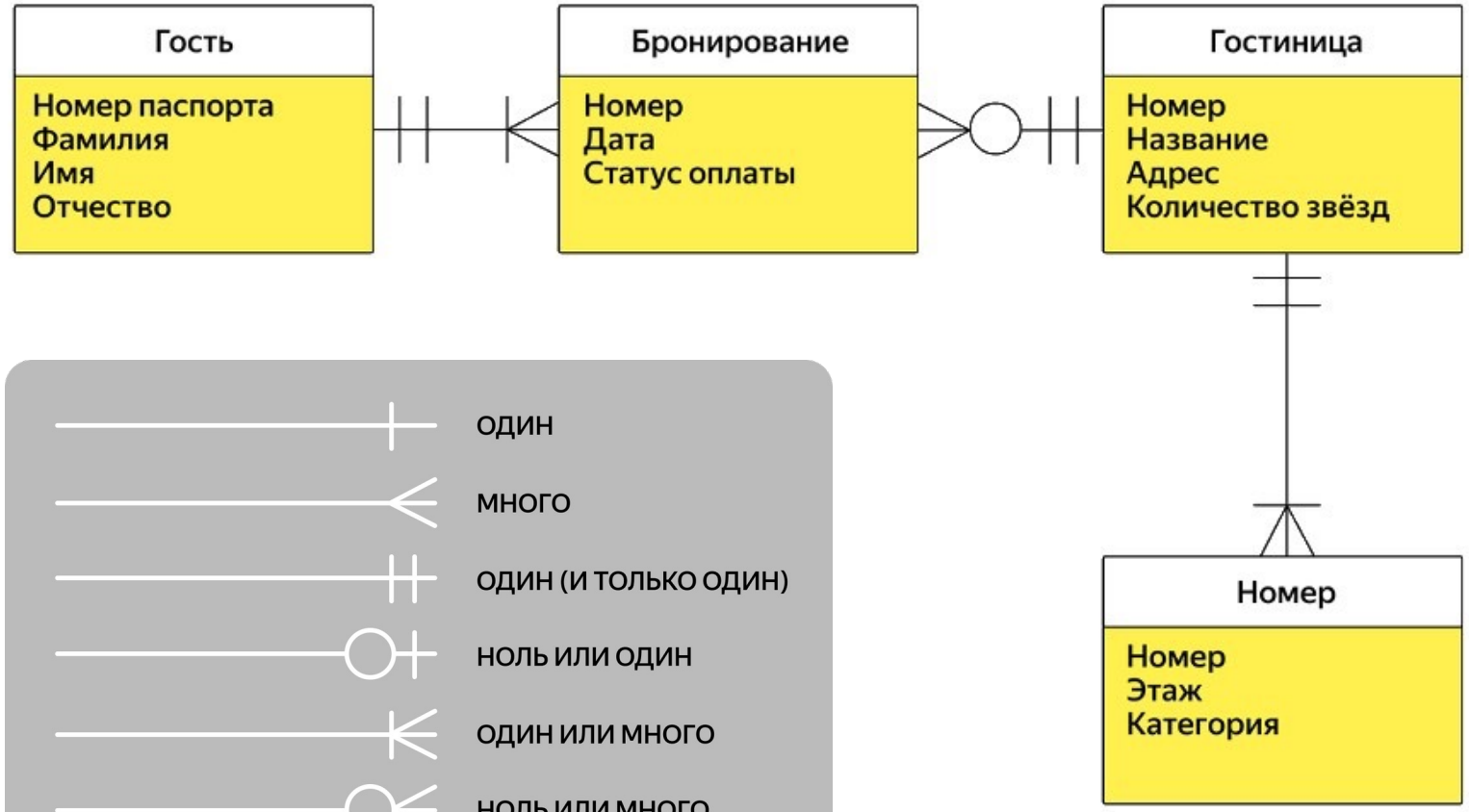




ER-диаграмма предметной области
Нотация Чена



ER-диаграмма предметной области
Нотация Мартина



Получение реляционной схемы из ER-диаграммы

1. Каждая простая **сущность превращается в таблицу** (отношение). Имя сущности становится именем таблицы.

2. **Связь «многие ко многим»** рассматривается как сущность-связь и **превращается в таблицу** (отношение).

3. Каждый **атрибут становится** возможным **столбцом** с тем же именем.

4. Компоненты уникального идентификатора сущности превращаются в **первичный ключ**. Если имеется несколько возможных уникальных идентификатора, выбирается наиболее используемый.

5. Связи «многие к одному» и «один к одному» становятся внешними ключами. Т.е. делается копия уникального идентификатора с конца связи «один», и соответствующие столбцы составляют **внешний ключ**.

6. **Создаются индексы** для первичного ключа (уникальный индекс), а также внешних ключей и тех атрибутов, которые будут часто использоваться в запросах.

12 правил Кодда (реляционные СУБД)

0. Основное правило (*Foundation Rule*)

Реляционная система управления базами данных (СУБД), должна быть способна управлять БД, используя исключительно свои реляционные возможности.

1. Информационное правило (*The Information Rule*)

Вся информация в реляционной базе данных на логическом уровне должна быть явно представлена единственным способом: значениями в таблицах.

2. Гарантированный доступ к данным (*Guaranteed Access Rule*)

В реляционной БД каждое отдельное (атомарное) значение данных должно быть логически доступно с помощью комбинации имени таблицы, значения первичного ключа и имени столбца.

12 правил Кодда (реляционные СУБД)

3. Систематическая поддержка NULL (Systematic Treatment of Null Values)

Неизвестные, или отсутствующие значения **NULL**, отличные от любого известного значения, должны поддерживаться для всех типов данных при выполнении любых операций. NULL не сводится к 0 для арифметических типов или к пустым строкам для символьных типов.

4. Доступ к каталогу данных в реляционной модели (Active On-Line Catalog Based on the Relational Model)

Схема БД должна представляться в форме реляционных таблиц с возможностью доступа к ним с использованием реляционной модели (по аналогии с пользовательскими таблицами).

5. Полнота подмножества языка (Comprehensive Data Sublanguage Rule)

Реляционная СУБД должна поддерживать хотя бы один реляционный язык, который:
(а) имеет линейный синтаксис,
(б) может использоваться как интерактивно, так и в прикладных программах,
(в) поддерживает операции определения структур данных, определения представлений, манипулирования данными, ограничения целостности, управления правами доступа и операции управления транзакциями.

12 правил Кодда (реляционные СУБД)

6. Возможность изменения представлений (View Updating Rule)

Каждое представление должно поддерживать все операции манипулирования данными, которые поддерживают реляционные таблицы: операции выборки, вставки, изменения и удаления данных.

7. Наличие высокоуровневых операций управления данными (High-Level Insert, Update, and Delete)

Операции вставки, изменения и удаления данных должны поддерживаться не только по отношению к одной строке реляционной таблицы, но и по отношению к любому множеству строк.

8. Физическая независимость данных (Physical Data Independence)

Приложения не должны зависеть от используемых способов хранения данных на носителях, от аппаратного обеспечения компьютеров, на которых находится реляционная БД.

12 правил Кодда (реляционные СУБД)

9. Логическая независимость данных (Logical Data Independence)

Представление данных в приложении не должно зависеть от структуры реляционных таблиц. Если в процессе нормализации одна реляционная таблица разделяется на две, представление должно обеспечить такое соединение этих данных, чтобы изменение структуры реляционных таблиц не сказывалось на работе приложений.

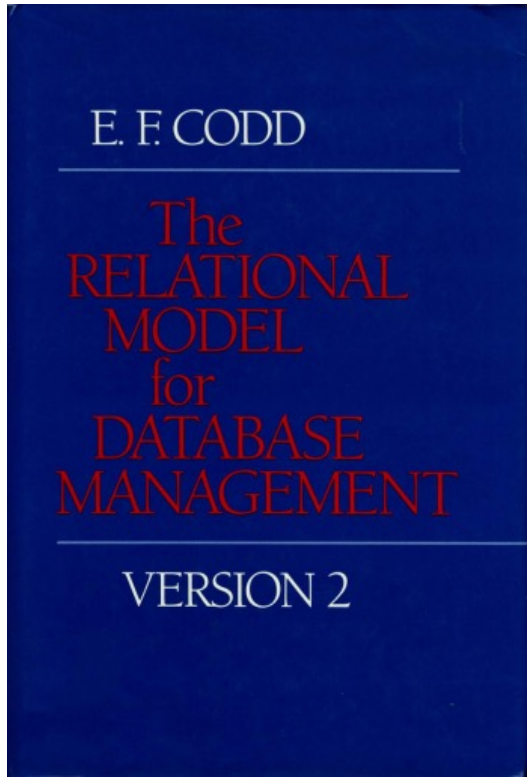
10. Независимость контроля целостности (Integrity Independence)

Вся информация, необходимая для поддержки целостности, должна находиться в каталоге данных (см. правило 4). Язык для работы с данными должен выполнять проверку входных данных и автоматически поддерживать целостность данных.

11. Независимость от расположения (Distribution Independence)

База данных может быть распределённой, может находиться на нескольких компьютерах, и это не должно оказывать влияния на приложения. Перенос базы данных на другой компьютер не должен оказывать влияния на приложения.

12 правил Кодда (реляционные СУБД)



Codd, Edgar F.
The relational model for database management: Version 2.
Addison-Wesley, 1990.
ISBN 9780201141924.

12. Согласование языковых уровней

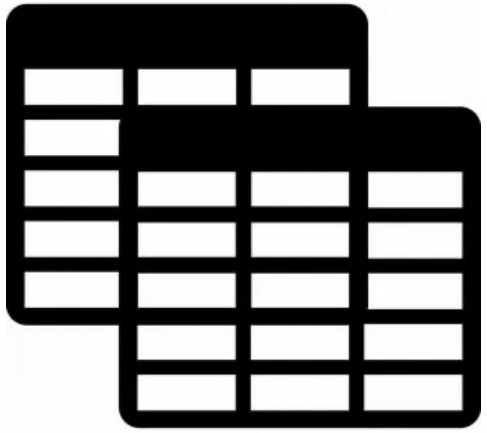
(The Nonsubversion Rule)

Если используется низкоуровневый язык доступа к данным, он не должен игнорировать правила безопасности и правила целостности, которые поддерживаются языком более высокого уровня.

Введение в PostgreSQL для пользователей SQL Server

Введение в PostgreSQL и его основные особенности

PostgreSQL — это свободно распространяемая объектно-реляционная система управления базами данных (СУБД) с открытым исходным кодом.







Официальная документация по PostgreSQL: https://www.postgresql.org/docs/

Сравнение

	PostgreSQL	SQL Server
Лицензирование и стоимость	СУБД с открытым исходным кодом	коммерческая СУБД, разработанная Microsoft
Поддержка платформы	работает в различных ОС, включая Windows, Linux, macOS и другие.	в основном работает в ОС Windows
Соответствие требованиям SQL	соответствие требованиям SQL и стандартам ANSI SQL	соответствие требованиям SQL и стандартам ANSI SQL некоторые проприетарные расширения и функции
Особенности и возможности	поддержка JSON, полнотекстовый поиск, массивы и т.п.	XML-данные, пространственные типы данных и возможности бизнес-аналитики (BI) с помощью служб SSAS и SSRS.

Сравнение

	PostgreSQL	SQL Server
Масштабируемость и производительность	высокая масштабируемость и производительность, поддерживает большие БД и приложения с высоким трафиком	
Репликация и высокая доступность	потоковая и логическую репликация	Always On Availability Groups и зеркальное отображение БД
Экосистема и инструментарий	богатая экосистема расширений, плагинов и инструментов	извлекает выгоду из экосистемы и инструментария Microsoft
Поддержка и документация	коммерческая поддержка доступна через сторонних поставщиков.	коммерческая поддержку через корпорацию Microsoft, включая техническую поддержку, обновления и исправления.

Установка и настройка соединения

Скачать PostgreSQL: https://www.postgresql.org/download/

Скачать pgAdmin: https://www.pgadmin.org/download/

Скачать Dbeaver: https://dbeaver.io/download/

Скачать Azure Data Studio: https://learn.microsoft.com/en-us/azure-data-studio/download-azure-data-studio?tabs=win-install%2Cwin-user-install%2Credhat-install%2Cwindows-uninstall%2Credhat-uninstall

5 Использование функций и агрегация данных

Введение во встроенные функции (Built-In Functions)

Типы функций	Описание
Скалярные	Оперируют одним кортежем, возвращают единственное значение
Логические	Скалярные функции, сравнивающие множество значений, чтобы получить результат
Агрегатные	Принимают одно или несколько входных значений и возвращают одно значение
Оконные	Действуют на окне (набор кортежей)
Табличные	Возвращают виртуальную таблицу (набор кортежей), которая может быть использована в дальнейших SQL-запросах

Скалярные функции (scalar)

- Работают со входными данными из одной строки
- Возвращают одно (скалярное) значение
- Можно использовать в выражениях в запросах
- Может быть детерминированной или недетерминированной (deterministic / non-deterministic)

Типы скалярных функций

- Configuration
- Conversion
- Cursor
- Date and Time
- Mathematical
- Metadata
- Security
- String
- System
- System Statistical
- Text and Image

Логические функции

Выход определяется с помощью сравнительной логики

- ISNUMERIC

```
SELECT ISNUMERIC('101.99') AS Is_a_Number;
```

- IIF

```
SELECT productid, listprice, IIF(listprice > 50, 'high','low') AS
PricePoint
FROM Production.Product;
```

- CHOOSE

```
SELECT ProductName, Color, Size,
        CHOOSE (ProductCategoryID,
'Bikes','Components','Clothing','Accessories') AS Category
FROM Production.Product;
```

Оконные функции

```
OVER (
    [ <PARTITION BY clause> ]
    [ <ORDER BY clause> ]
    [ <ROW or RANGE clause> ]
   )

<PARTITION BY clause> ::=
PARTITION BY value_expression , ... [ n ]
```

```
<ORDER BY clause> ::=
ORDER BY order_by_expression
   [ COLLATE collation_name ]  [ ASC | DESC ]  [ ,...n ]

<ROW or RANGE clause> ::=
{ ROWS | RANGE } <window frame extent>

<window frame extent> ::=  {
  <window frame preceding>  | <window frame between>
}
<window frame between> ::=
 BETWEEN <window frame bound> AND <window frame
bound>
```

```sql
-- все строки + PARTITION
SELECT ProductCategoryID, ProductID, Name, ListPrice,
AVG(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN
        unbounded preceding and unbounded following) AS AvgPrice,
Count(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN
        unbounded preceding and unbounded following) AS CountPrice,
Sum(ListPrice) OVER(PARTITION BY ProductCategoryID ORDER BY ListPrice asc ROWS BETWEEN
        unbounded preceding and unbounded following) AS SumPrice
FROM SalesLT.Product AS p
ORDER BY ProductCategoryID asc;
```

Оконные функции

- Функции применяются к окну – набору строк
- Включают функции ранжирования, смещения, агрегации и распределения

```
SELECT TOP(3) ProductID, Name, ListPrice,
      RANK() OVER(ORDER BY ListPrice DESC) AS RankByPrice
FROM Production.Product
ORDER BY RankByPrice;
```



ProductID	ProductName	UnitPrice	RankByPrice
8	Gizmo	263.50	1
29	Widget	123.79	2
9	Thingybob	97.00	3

Агрегатные функции

- Функции, которые оперируют на наборе или множестве строк
- «Обобщают» строки
- Без предложения GROUP BY, все строки рассматриваются как одна группа

```
SELECT COUNT(*) AS OrderLines,
       SUM(OrderQty*UnitPrice) AS TotalSales
FROM    Sales.OrderDetail;
```



OrderLines	TotalSales
542	714002.9136

Группировка с предложением GROUP BY

- GROUP BY создает группы для строк с результатом на основе уникальных комбинаций значений, определенных в предложении GROUP BY
- GROUP BY вычисляет обобщающие значения для агрегатных функций на последующих фазах
- Исходные строки с детализацией «теряются» после применения предложения GROUP BY

```
SELECT CustomerID, COUNT(*) AS Orders
FROM Sales.SalesOrderHeader
GROUP BY CustomerID;
```

Фильтрация с предложением HAVING

- Предложение HAVING накладывает условие, которому должна удовлетворять каждая группа
- Предложение WHERE обрабатывается перед GROUP BY, предложение HAVING обрабатывается после GROUP BY

```
SELECT CustomerID, COUNT(*) AS Orders
FROM Sales.SalesOrderHeader
GROUP BY CustomerID
HAVING COUNT(*) > 10;
```

6 Использование подзапросов и предложения APPLY

Введение в подзапросы

- Подзапросы являются вложенными запросами: запросы в запросах
- Результаты вложенного запроса передаются внешнему запросу
  - Вложенный запрос выступает в роли выражения с точки зрения внешнего запроса



Скалярные и многозначные подзапросы

- Скалярный (однозначный) подзапрос возвращает одно значение во внешний запрос
  - Можно использовать везде, где можно использовать однозначные выражения: SELECT, WHERE и т.д.
- Многозначные подзапросы возвращают многозначные значения в виде одного столбца внешнему запросу
  - Используется с предикатом IN

```
SELECT orderid, productid, unitprice, qty
FROM Sales.OrderDetails
WHERE orderid =
      (SELECT MAX(orderid) AS lastorder
      FROM Sales.Orders);
```

```
SELECT custid, orderid
FROM Sales.orders
WHERE custid IN (
      SELECT custid
      FROM Sales.Customers
      WHERE countryregion = N'Mexico');
```

Замкнутые и коррелированные подзапросы

- Большинство подзапросов замкнуты, т.е. не имеют взаимосвязи с внешним запросом помимо передачи результата
- Коррелированные подзапросы ссылаются на сущности (столбцы таблиц) внешнего запроса
  - Зависят от внешнего запроса, нельзя исполнять отдельно
  - Поведение подзапроса: один запуск для каждой строки внешнего запроса
  - Могут возвращать скалярное или множественное значение

```
SELECT orderid, empid, orderdate
FROM Sales.Orders AS O1
WHERE orderdate = (SELECT MAX(orderdate)
                   FROM Sales.Orders AS O2
                   WHERE O2.empid = O1.empid)
ORDER BY empid, orderdate;
```

Использование предложения APPLY с табличными функциями

- CROSS APPLY применяет табличное выражение справа к каждой строке в левой таблице
  - Концептуально похоже на CROSS JOIN между двумя таблицами, но может сопоставлять данные между источниками

```
SELECT S.supplierid, s.companyname, P.productid,    P.productname, P.unitprice
FROM Production.Suppliers AS S
CROSS APPLY dbo.fn_TopProductsByShipper(S.supplierid) AS P
```

- OUTER APPLY добавляет строки со значениями NULL для правой таблицы
  - Концептуально похоже на LEFT OUTER JOIN между двумя таблицами